



รายงานโครงงาน

กลุ่มที่ 12

ระบบส่งคืนของหาย Lost and Found Lost and Found Management System

โดย

นางสาว	ณัฐนันท์	บุษดี	รหัสนักศึกษา	673380037-1
นางสาว	ศิริรัตน์	ชัยชนะ	รหัสนักศึกษา	673380060-6
นางสาว	ปภาวรินทร์	นาเมืองรักษ์	รหัสนักศึกษา	673380275-5
นางสาว	ปรนันท	บุสดีวงศ์	รหัสนักศึกษา	673380276-3
นางสาว	วิภาวี	ฤทธิหาญ	รหัสนักศึกษา	673380514-3

เสนอ

รศ.ดร.ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353002 Principles of Software Design and
Development

ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

บทคัดย่อ

โครงการนี้มีวัตถุประสงค์เพื่อออกแบบและพัฒนาระบบแจ้งของหายและติดตามการส่งคืนของที่พบ (Lost and Found System) โดยใช้สถาปัตยกรรมแบบ Layered Architecture ร่วมกับเฟรมเวิร์ก Spring Boot 3.4.3 บนภาษา Java 17 เชื่อมต่อกับฐานข้อมูลเชิงสัมพันธ์ PostgreSQL ผ่าน Spring Data JPA และให้บริการผ่าน RESTful API ที่มีเอกสารประกอบด้วย Swagger/OpenAPI ส่วนฝั่งผู้ใช้งานพัฒนาด้วย React (Vite) เชื่อมต่อการยืนยันตัวตนทั้งแบบอีเมล/รหัสผ่านและผ่าน Firebase Authentication

ในการออกแบบระบบได้ประยุกต์ใช้หลักการ SOLID Principles และ Design Pattern ตามแนวทางของ Gang of Four (Gamma, Helm, Johnson, & Vlissides, 1994) จำนวน 3 รูปแบบในกลุ่ม Behavioral Pattern ได้แก่ State Pattern สำหรับควบคุมการเปลี่ยนสถานะของประกาศ (Report) Observer Pattern สำหรับการแจ้งเตือนผู้ติดตามประกาศโดยอัตโนมัติ และ Strategy Pattern สำหรับตรวจสอบสิทธิ์ การยื่นคำขอเคลมที่มีเงื่อนไขแตกต่างกันตามประเภทของประกาศ นอกจากนี้ยังออกแบบฐานข้อมูล โดยใช้กลยุทธ์รหัสหลักแบบผสม (Hybrid ID Strategy) ซึ่งใช้ UUID สำหรับตารางที่ผู้ใช้เข้าถึงผ่าน URL โดยตรง และใช้ BIGINT สำหรับตารางข้อมูลหลักและตารางภายใน เพื่อความสมดุลระหว่างความปลอดภัยและประสิทธิภาพ

ผลการดำเนินโครงการพบว่าระบบสามารถทำงานได้ครบถ้วนตามขอบเขตที่กำหนด ทั้งการแจ้งของหาย/พบของ การค้นหาและติดตามประกาศ การยื่นและอนุมัติคำขอเคลม การแจ้งเตือนอัตโนมัติ และการจัดการข้อมูลโดยผู้ดูแลระบบ ระบบถูก Deploy ขึ้นบน Cloud จริงผ่าน Docker Container และสามารถเข้าถึงได้ผ่าน URL สาธารณะ ผลการพัฒนานี้แสดงให้เห็นถึงความสามารถในการประยุกต์ใช้หลักการออกแบบซอฟต์แวร์เชิงวัตถุและสถาปัตยกรรมซอฟต์แวร์สมัยใหม่ในการแก้ปัญหาจริง

คำสำคัญ: Spring Boot, Layered Architecture, SOLID Principles, Design Pattern, RESTful API, Lost and Found System

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อประกอบการนำเสนอโครงการในรายวิชา CP353002 Principles of Software Design and Development โดยมีเนื้อหาครอบคลุมตั้งแต่ที่มาและความสำคัญของปัญหา แนวคิดทฤษฎีที่เกี่ยวข้อง วิธีดำเนินการพัฒนาระบบ ผลการดำเนินงาน ตลอดจนการสรุปผล และข้อเสนอแนะสำหรับการพัฒนาต่อในอนาคต

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจศึกษาการประยุกต์ใช้หลักการออกแบบซอฟต์แวร์ (Software Design Principles) และรูปแบบการออกแบบ (Design Patterns) ในการพัฒนาระบบจริงด้วยสถาปัตยกรรมแบบ Layered Architecture รวมถึงเป็นแนวทางสำหรับการพัฒนาระบบที่มีลักษณะใกล้เคียงกันต่อไป ขอขอบคุณอาจารย์ประจำรายวิชาที่ได้ให้คำแนะนำและข้อเสนอแนะอันเป็นประโยชน์ตลอดระยะเวลาการดำเนินโครงการ หากรายงานฉบับนี้มีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี้

คณะผู้จัดทำ

สารบัญ

เรื่อง	หน้า
บทคัดย่อ	ก
คำนำ	ข
สารบัญ	ค
สารบัญตาราง	จ
สารบัญภาพ	ฉ
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของโครงการ	1
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
บทที่ 2 เอกสารและงานวิจัยที่เกี่ยวข้อง	3
2.1 สถาปัตยกรรมแบบ Layered Architecture	3
2.2 หลักการ SOLID Principles	3
2.3 รูปแบบการออกแบบ (Design Patterns)	3
2.4 สถาปัตยกรรมแบบ RESTful API	4
2.5 การจัดการข้อมูลเชิงวัตถุ-สัมพันธ์ (Object-Relational Mapping)	4
2.6 งานและระบบที่เกี่ยวข้อง	5
2.7 การทดสอบซอฟต์แวร์ (Software Testing)	5
บทที่ 3 วิธีดำเนินการ	6
3.1 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา	6
3.2 การออกแบบสถาปัตยกรรมระบบ	7
3.3 การออกแบบฐานข้อมูล	9
3.4 การประยุกต์ใช้หลักการ SOLID	10
3.5 การประยุกต์ใช้ Design Pattern	11
3.6 การพัฒนา RESTful API	13
3.7 การบริหารจัดการทีมและ Git Workflow	16
3.8 การทดสอบและการนำระบบขึ้นใช้งานจริง (Deployment)	16
บทที่ 4 ผลการดำเนินการ	18

เรื่อง	หน้า
4.1 ผลการออกแบบฐานข้อมูล	18
4.2 ผลการพัฒนา RESTful API	18
4.3 ผลการจัดทำแผนภาพระบบ (Diagrams)	19
4.4 ผลการทดสอบระบบ	20
4.5 ผลการนำระบบขึ้นใช้งานจริง	26
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ	27
5.1 สรุปผลการดำเนินโครงการ	27
5.2 อภิปรายผล	27
5.3 ข้อเสนอแนะ	28
บรรณานุกรม	30
ภาคผนวก	31
ภาคผนวก ก รายการ RESTful API Endpoint ทั้งหมด	31
ภาคผนวก ข ผลการทดสอบ API ทั้ง 22 กรณี	33

สารบัญตาราง

ตาราง	หน้า
ตารางที่ 1 เครื่องมือและเทคโนโลยีที่ใช้พัฒนาระบบ	6
ตารางที่ 2 สมาชิกกลุ่มและหน้าที่รับผิดชอบ	6
ตารางที่ 3 สรุปหลักการ SOLID และตำแหน่งที่ปรากฏในโค้ด	10
ตารางที่ 4 Enterprise / Architectural Pattern ที่ใช้ในระบบ	11
ตารางที่ 5 GoF Design Pattern และเหตุผลการเลือกใช้	11
ตารางที่ 6 รายชื่อตารางในฐานข้อมูลทั้งหมด	18
ตารางที่ 7 ตัวอย่าง RESTful API Endpoint หลัก	19
ตารางที่ 8 สรุปผลการทดสอบระบบ	20
ตารางที่ 9 ผลการวัดความครอบคลุมของการทดสอบ (Code Coverage)	21
ตารางที่ 10 ผลการทดสอบ Unit Test ของ ClaimServiceImpl	22
ตารางที่ 11 ผลการทดสอบ Integration Test	23
ตารางที่ 12 สรุปผลการทดสอบ API แยกตามกลุ่มทรัพยากร	23
ตารางที่ 13 ผลการทดสอบการยืนยันตัวตนและสิทธิ์การเข้าถึง	24
ตารางที่ 14 ผลการทดสอบฟังก์ชันของ Frontend	25
ตารางที่ 15 รายงานข้อบกพร่องที่พบจากการทดสอบ	25
ตารางที่ 16 รายการ RESTful API Endpoint ทั้งหมด	31
ตารางที่ 17 ผลการทดสอบ API ทั้ง 22 กรณี	33

สารบัญภาพ

ภาพ	หน้า
ภาพที่ 1 Component Diagram	8
ภาพที่ 2 Deployment Diagram	8
ภาพที่ 3 Entity Relationship (ER) Diagram	9
ภาพที่ 4 Domain Model (Conceptual Class Diagram)	10
ภาพที่ 5 Class Diagram พร้อมตำแหน่ง Design Pattern	12
ภาพที่ 6 State Diagram (Report Status)	12
ภาพที่ 7 State Diagram (Claim Status)	13
ภาพที่ 8 Sequence Diagram	14
ภาพที่ 9 Use Case Diagram	14
ภาพที่ 10 Activity Diagram	15
ภาพที่ 11 ผลการรันชุดทดสอบอัตโนมัติด้วยคำสั่ง mvn test	21
ภาพที่ 12 ผลการรัน Unit Test ของ ClaimServiceImpl	22
ภาพที่ 13 ผลการรัน Integration Test (ClaimFlowIntegrationTest)	23
ภาพที่ 14 ตัวอย่างผลการทดสอบ API ผ่าน Swagger UI	24
ภาพที่ 15 ภาพหน้าจอข้อบกพร่อง BUG-01 บนโทรศัพท์มือถือ Android	26

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของปัญหา

ของหายเป็นปัญหาที่พบได้ทั่วไปในสถานศึกษาและองค์กรขนาดใหญ่ เมื่อผู้ใช้งานทำสิ่งของสูญหาย มักไม่มีช่องทางกลางที่เชื่อถือได้สำหรับแจ้งเตือน หรือติดตามสถานะของสิ่งของดังกล่าว ในขณะเดียวกันผู้ที่พบสิ่งของก็ไม่มีช่องทางที่สะดวก ในการแจ้งให้เจ้าของทราบ ส่งผลให้ของหายจำนวนมากไม่ได้ถูกส่งคืนแก่เจ้าของอย่างแท้จริง อีกทั้งกระบวนการพิสูจน์ความเป็นเจ้าของยังขาดความโปร่งใสและตรวจสอบย้อนหลังได้ยาก หากดำเนินการผ่านช่องทางที่ไม่เป็นระบบ เช่น กลุ่มสนทนาในสื่อสังคมออนไลน์ หรือกระดาษประกาศ

คณะผู้จัดทำจึงเล็งเห็นความสำคัญของการพัฒนาระบบสารสนเทศที่ช่วยให้การแจ้งของหาย การค้นหา การยื่นคำขอเคลม และการอนุมัติการส่งคืนเป็นไปอย่างมีระบบ มีการบันทึกประวัติ การเปลี่ยนแปลงสถานะที่ตรวจสอบได้ และสามารถแจ้งเตือนผู้ที่เกี่ยวข้องโดยอัตโนมัติ โดยใช้โอกาสนี้ในการประยุกต์ใช้องค์ความรู้ด้านหลักการออกแบบซอฟต์แวร์ (Software Design Principles) และสถาปัตยกรรมซอฟต์แวร์ (Software Architecture) ที่ได้ศึกษาในรายวิชา CP353002 มาพัฒนาเป็นระบบที่ใช้งานได้จริง

1.2 วัตถุประสงค์ของโครงการ

1. เพื่อพัฒนาระบบแจ้งของหายและติดตามการส่งคืนของที่พบ ให้ผู้ที่ทำของหายและผู้พบของมีช่องทางกลางในการติดต่อกันได้อย่างสะดวก
2. เพื่ออำนวยความสะดวกแก่ผู้ใช้งานในการค้นหาประกาศ ติดตามสถานะ และรับการแจ้งเตือนอัตโนมัติเมื่อมีความคืบหน้า
3. เพื่อให้กระบวนการยื่นคำขอเคลมและพิสูจน์ความเป็นเจ้าของมีความโปร่งใส และสามารถตรวจสอบย้อนหลังได้
4. เพื่อเพิ่มโอกาสที่สิ่งของที่สูญหายจะได้รับการส่งคืนแก่เจ้าของอย่างถูกต้อง
5. เพื่อให้ผู้ดูแลระบบสามารถจัดการข้อมูลและประกาศที่ไม่เหมาะสมได้อย่างเป็นระบบ

1.3 ขอบเขตของโครงการ

ระบบที่พัฒนาครอบคลุมการทำงานหลัก ได้แก่ การสมัครสมาชิกและเข้าสู่ระบบทั้งแบบอีเมล/รหัสผ่านและผ่าน Firebase (Google Login) การแจ้งประกาศของหาย/พบของ

พร้อมแนบรูปภาพ การค้นหาและกรองประกาศ การติดตามประกาศ (Watch) การยื่นคำขอเคลมพร้อมหลักฐาน การอนุมัติ/ปฏิเสธคำขอเคลมโดยเจ้าของประกาศ การแจ้งเตือนอัตโนมัติเมื่อสถานะประกาศเปลี่ยนแปลง และการจัดการข้อมูลโดยผู้ดูแลระบบ (Staff) เช่น การลบประกาศที่ไม่เหมาะสม ระบบพัฒนาด้วย Spring Boot 3.4.3 สำหรับฝั่ง Backend และ React (Vite) สำหรับฝั่ง Frontend เชื่อมต่อฐานข้อมูล PostgreSQL บน Neon Cloud และนำขึ้นใช้งานจริงผ่าน Docker Container บน Render

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ผู้ที่ทำของหายมีช่องทางที่เชื่อถือได้ในการแจ้งและติดตามสิ่งของ และมีโอกาสได้รับของคืนมากขึ้น
2. ผู้ที่พบสิ่งของสามารถแจ้งให้เจ้าของทราบและส่งคืนสิ่งของได้อย่างสะดวกและถูกต้อง
3. เจ้าของประกาศและผู้ยื่นคำขอเคลมได้รับกระบวนการพิสูจน์ความเป็นเจ้าของที่โปร่งใส และมีประวัติการเปลี่ยนแปลงสถานะที่ตรวจสอบได้
4. ผู้ดูแลระบบและสถานศึกษา/องค์กรสามารถบริหารจัดการข้อมูลของหายได้อย่างเป็นระบบ ลดภาระการประสานงานผ่านกลุ่มสนทนาหรือกระดานประกาศ

บทที่ 2

เอกสารและงานวิจัยที่เกี่ยวข้อง

2.1 สถาปัตยกรรมแบบ Layered Architecture

Layered Architecture เป็นรูปแบบสถาปัตยกรรมซอฟต์แวร์ที่แบ่งระบบออกเป็นชั้น (Layer) ตามหน้าที่ความรับผิดชอบ โดยแต่ละชั้นสื่อสารกับชั้นที่อยู่ติดกันเท่านั้น ทำให้ระบบมีความเป็นระเบียบง่ายต่อการบำรุงรักษา และสามารถเปลี่ยนแปลงแต่ละชั้นได้โดยไม่กระทบชั้นอื่นมากนัก Fowler (2002) ได้อธิบายรูปแบบสถาปัตยกรรมระดับองค์กร (Enterprise Application Architecture Pattern) ไว้หลายรูปแบบที่สอดคล้องกับแนวคิดนี้ เช่น Layered Architecture, Service Layer, Repository Pattern, Data Transfer Object (DTO) และ Model-View-Controller (MVC) ซึ่งล้วนเป็นรูปแบบที่นิยมใช้ในการพัฒนาเว็บ แอปพลิเคชันระดับองค์กรด้วยเฟรมเวิร์กเช่น Spring Boot

2.2 หลักการ SOLID Principles

SOLID เป็นตัวย่อของหลักการออกแบบซอฟต์แวร์เชิงวัตถุ 5 ประการที่ Martin (2003) ได้รวบรวมและเผยแพร่ไว้ในหนังสือ Agile Software Development: Principles, Patterns, and Practices ได้แก่ Single Responsibility Principle (SRP) ที่ระบุว่าแต่ละคลาสควรมีหน้าที่ความรับผิดชอบเพียงหนึ่งเดียว Open/Closed Principle (OCP) ที่ระบุว่าโมดูลควรเปิดให้ขยายได้ แต่ปิดการแก้ไข Liskov Substitution Principle (LSP) ที่ระบุว่าคลาสลูกต้องสามารถใช้แทนคลาสแม่ได้โดยไม่ทำให้ตรรกะของโปรแกรมผิดพลาด Interface Segregation Principle (ISP) ที่ระบุว่าไม่ควรบังคับให้คลาสขึ้นกับอินเทอร์เฟซที่ตนเองไม่ได้ใช้งาน และ Dependency Inversion Principle (DIP) ที่ระบุว่าโมดูลระดับสูงไม่ควรขึ้นกับโมดูลระดับต่ำโดยตรง แต่ควรขึ้นกับนามธรรม (Abstraction) ร่วมกัน หลักการเหล่านี้เป็นรากฐานสำคัญของการออกแบบซอฟต์แวร์ ที่ยืดหยุ่นและบำรุงรักษาง่าย

2.3 รูปแบบการออกแบบ (Design Patterns)

Design Pattern คือแนวทางการแก้ปัญหาการออกแบบซอฟต์แวร์ที่เกิดขึ้นซ้ำ ๆ ซึ่งผ่านการพิสูจน์แล้วว่าใช้ได้ผลดี Gamma, Helm, Johnson และ Vlissides (1994) หรือที่รู้จักกันในนาม "Gang of Four (GoF)" ได้รวบรวมรูปแบบการออกแบบไว้ 23 รูปแบบ แบ่งออกเป็น 3 กลุ่มหลัก ได้แก่ Creational Pattern (เกี่ยวกับการสร้างวัตถุ) Structural Pattern (เกี่ยวกับโครงสร้างความสัมพันธ์ระหว่างคลาส) และ Behavioral Pattern (เกี่ยวกับพฤติกรรม)

และการสื่อสารระหว่างวัตถุ) โครงการนี้เลือกประยุกต์ใช้รูปแบบในกลุ่ม Behavioral Pattern จำนวน 3 รูปแบบ ได้แก่

1. State Pattern: อนุญาตให้วัตถุเปลี่ยนพฤติกรรมได้เมื่อสถานะภายในเปลี่ยนแปลง โดยห่อหุ้มพฤติกรรมของแต่ละสถานะไว้ในคลาสของตนเอง
2. Observer Pattern: กำหนดความสัมพันธ์แบบหนึ่งต่อหลายระหว่างวัตถุ เมื่อวัตถุหนึ่งเปลี่ยนสถานะ วัตถุที่ขึ้นต่อกันทั้งหมดจะได้รับการแจ้งเตือนโดยอัตโนมัติ
3. Strategy Pattern: กำหนดกลุ่มอัลกอริทึมที่สามารถสลับใช้งานแทนกันได้ในการทำงาน (Runtime) โดยไม่กระทบกับโค้ดส่วนที่เรียกใช้งาน

การเลือกใช้รูปแบบในกลุ่ม Behavioral Pattern สอดคล้องกับลักษณะปัญหาของระบบที่เกี่ยวข้องกับการเปลี่ยนสถานะของประกาศ การแจ้งเตือนผู้ใช้งาน และการตรวจสอบเงื่อนไขที่แตกต่างกันตามประเภทของข้อมูล

2.4 สถาปัตยกรรมแบบ RESTful API

Representational State Transfer (REST) เป็นรูปแบบสถาปัตยกรรมสำหรับระบบแบบกระจาย (Distributed System) ที่ Fielding (2000) นำเสนอไว้ในวิทยานิพนธ์ปริญญาเอก โดยมีหลักการสำคัญ เช่น การสื่อสารแบบ Stateless การระบุทรัพยากร (Resource) ด้วย Uniform Resource Identifier (URI) และการใช้ HTTP Method ตามความหมายมาตรฐาน (GET, POST, PUT, PATCH, DELETE) REST ได้กลายเป็นมาตรฐานโดยพฤตินัยสำหรับการออกแบบ Web API ในปัจจุบัน และถูกนำมาประยุกต์ใช้ในการออกแบบ API ของระบบนี้ โดยตั้งชื่อ Endpoint แบบอิงทรัพยากร (Resource-based) เช่น /api/v1/reports/{id}/claims และใช้ HTTP Status Code ตามความหมายมาตรฐาน

2.5 การจัดการข้อมูลเชิงวัตถุ-สัมพันธ์ (Object-Relational Mapping)

Object-Relational Mapping (ORM) เป็นเทคนิคที่ใช้แปลงข้อมูลระหว่างระบบเชิงวัตถุ (Object-Oriented) กับฐานข้อมูลเชิงสัมพันธ์ (Relational Database) โดย Java Persistence API (JPA) เป็นมาตรฐานของภาษา Java สำหรับการทำ ORM และ Hibernate เป็นหนึ่งใน Implementation ที่ได้รับความนิยมสูงสุด Spring Data JPA เป็นส่วนขยายของ Spring Framework ที่ช่วยลดโค้ดซ้ำซ้อนในการเข้าถึงข้อมูล (Repository Pattern) โดยอัตโนมัติ ซึ่งโครงการนี้ได้นำมาใช้ร่วมกับฐานข้อมูล PostgreSQL ในการจัดการข้อมูลทั้งหมด 12 ตาราง

2.6 งานและระบบที่เกี่ยวข้อง

ระบบ Traffy Fondue ซึ่งพัฒนาโดยสำนักงานพัฒนาวิทยาศาสตร์และเทคโนโลยีแห่งชาติ (สวทช.) เป็นตัวอย่างระบบแก้ปัญหามือถือที่มีแนวคิดใกล้เคียงกับระบบที่พัฒนาในโครงการนี้ กล่าวคือ มีการแจ้งปัญหา การติดตามสถานะ และการแจ้งเตือนเมื่อปัญหาได้รับการแก้ไข คณะผู้จัดทำ ได้นำแนวคิดเรื่องการติดตามสถานะ (Status Tracking) และการแจ้งเตือนอัตโนมัติ มาประยุกต์ใช้ในการออกแบบ State Pattern และ Observer Pattern ของระบบ Lost and Found นี้

2.7 การทดสอบซอฟต์แวร์ (Software Testing)

การทดสอบซอฟต์แวร์เป็นกระบวนการตรวจสอบว่าระบบทำงานถูกต้องตามความต้องการ และช่วยตรวจพบข้อผิดพลาดก่อนนำระบบไปใช้งานจริง แนวคิดที่นิยมใช้วางสัดส่วนการทดสอบอัตโนมัติคือพีระมิดการทดสอบ (Test Pyramid) ซึ่ง Cohn (2009) เสนอไว้เป็นครั้งแรก และ Fowler (2012b) นำมาอธิบายขยายความ แนวคิดนี้แบ่งการทดสอบเป็นสามชั้น ชั้นฐานคือ Unit Test จำนวนมากที่สุดที่รันเร็วและมีต้นทุนต่ำ ชั้นกลางคือ Integration หรือ Service Test ที่ทดสอบการทำงานร่วมกันของหลายส่วน และชั้นยอดคือการทดสอบผ่านส่วนติดต่อผู้ใช้ (UI/End-to-End) ที่ควรมีจำนวนน้อยที่สุด เพราะรันช้า เปราะบาง และมีต้นทุนสูง

Unit Test ทดสอบหน่วยโค้ดขนาดเล็กโดยแยกออกจากส่วนอื่น โครงการนี้ใช้ JUnit 5 เป็นเฟรมเวิร์กการทดสอบ (JUnit Team, n.d.) ร่วมกับ Mockito ที่สร้างวัตถุจำลอง (Mock Object) แทนส่วนที่ Service พึ่งพา เช่น Repository (Mockito, n.d.) ทำให้ทดสอบ Business Logic ได้โดยไม่ต้องเชื่อมต่อกับฐานข้อมูลจริง ซึ่งเป็นผลดีจากการที่ระบบออกแบบให้ Service ขึ้นกับ Interface ตามหลัก Dependency Inversion

Integration Test ตรวจสอบการทำงานร่วมกันของหลายส่วนกับโครงสร้างพื้นฐานจริง Testcontainers เป็นไลบรารีที่สร้าง Docker Container ชั่วคราวระหว่างการทดสอบ (Testcontainers, n.d.) จึงทดสอบกับ PostgreSQL ชนิดเดียวกับระบบจริงได้ แทนการใช้ฐานข้อมูลจำลองที่พฤติกรรมอาจต่างไป เช่น Partial Unique Index ที่ระบบนี้ใช้ป้องกันการยื่นคำขอเคลมซ้ำ

ความครอบคลุมของการทดสอบ (Code Coverage) คือสัดส่วนของโค้ดที่ถูกรันโดยชุดทดสอบ วัดได้หลายมิติ ได้แก่ Instruction, Branch, Line, Method และ Class โดยเครื่องมือที่ใช้คือ JaCoCo (JaCoCo, n.d.) อย่างไรก็ตาม Fowler (2012a) ชี้ว่า Coverage เป็นเครื่องมือที่มีประโยชน์ในการค้นหาส่วนที่ยังไม่ถูกทดสอบ แต่ตัวเลขเปอร์เซ็นต์เพียงอย่างเดียวบอกคุณภาพของชุดทดสอบได้น้อย จึงควรใช้ควบคู่กับการออกแบบกรณีทดสอบที่มีความหมายต่อกฎทางธุรกิจ

บทที่ 3

วิธีดำเนินการ

3.1 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา

คณะผู้จัดทำได้เลือกใช้เครื่องมือและเทคโนโลยี ดังตารางที่ 1 โดยพิจารณา จากความเหมาะสมกับขอบเขตของโครงการและข้อกำหนดทางเทคนิคของรายวิชา

ตารางที่ 1 เครื่องมือและเทคโนโลยีที่ใช้พัฒนาระบบ

หมวดหมู่	เทคโนโลยีที่ใช้
Backend Framework	Spring Boot 3.4.3 (Java 17)
Build Tool	Maven
Database	PostgreSQL (Neon Cloud)
ORM	Spring Data JPA (Hibernate)
Authentication	Spring Security + JWT, Firebase Authentication (Google Login)
API Documentation	Swagger / OpenAPI (Springdoc)
Frontend	React (Vite)
Containerization	Docker, Docker Compose
Deployment	Render
Version Control	Git, GitHub

โดยสมาชิกในกลุ่มได้แบ่งหน้าที่ความรับผิดชอบดังตารางที่ 2

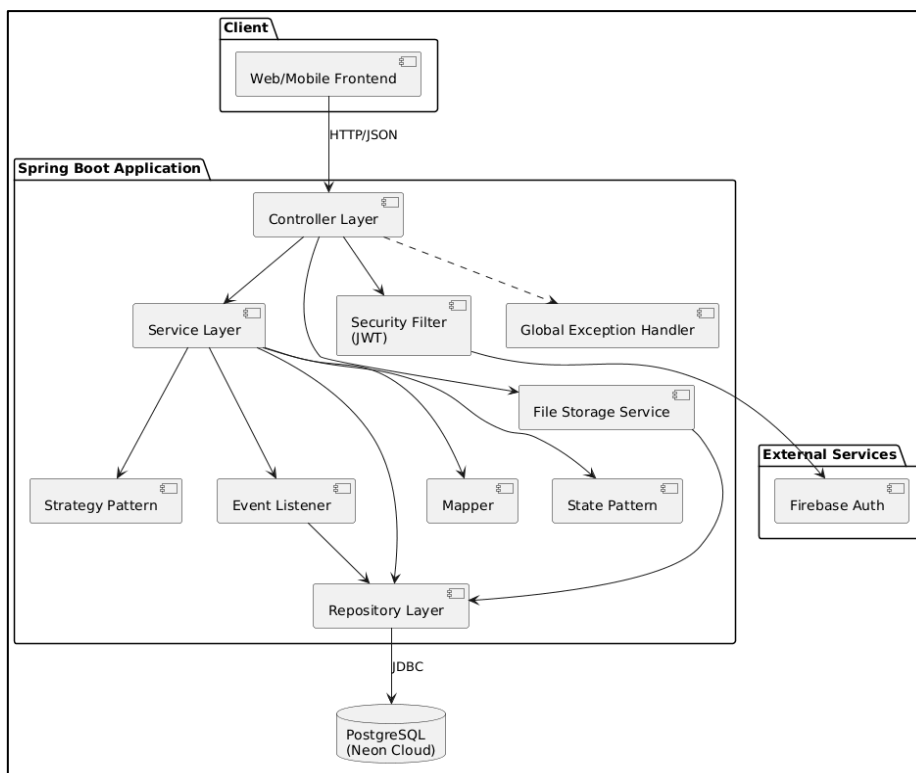
ตารางที่ 2 สมาชิกกลุ่มและหน้าที่รับผิดชอบ

ลำดับ	ชื่อ-นามสกุล	หน้าที่รับผิดชอบ
1	นางสาวณัฐนันท์ บุชดี	Backend Data Layer & Document Master: Database Schema, Entity, Spring Data JPA Repository, คู่มือภาพรวมเอกสารและ Diagrams

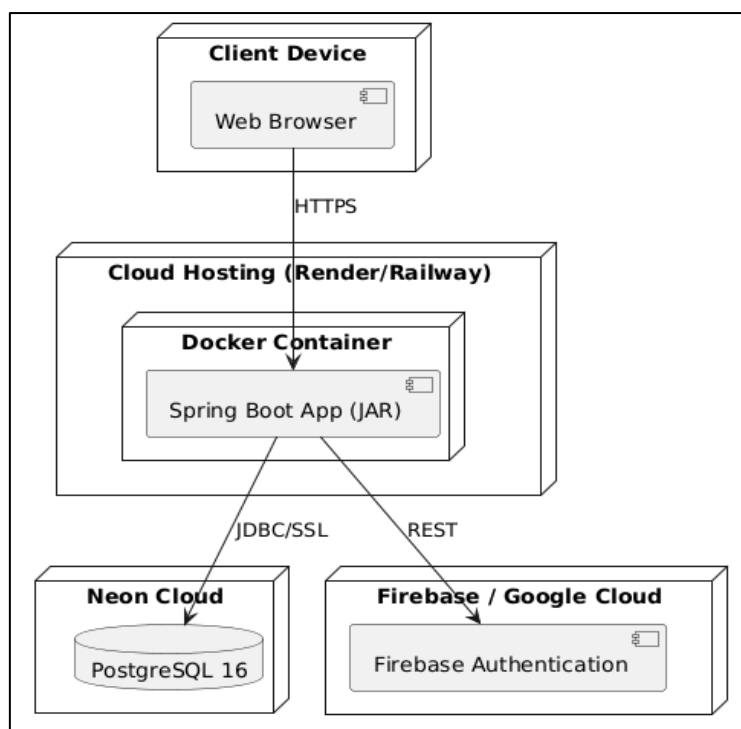
ลำดับ	ชื่อ-นามสกุล	หน้าที่รับผิดชอบ
2	นางสาวศิริรัตน์ ชัยชนะ	Core Backend Logic & API Service: REST Controllers, Service Layer, DTO/Mapper, Validation, Swagger/OpenAPI
3	นางสาวปภาวรินทร์ นาเมืองรักษ์	System Architecture & Cloud/DevOps: System Boilerplate, Neon Cloud DB, Dockerfile, docker-compose.yml, Cloud Deployment
4	นางสาวปรนันท บุษดีวงศ์	QA Specialist: API & System Testing, JUnit 5/Mockito Tests
5	นางสาววิภาวี ฤทธิหาญ	Frontend Lead Developer: Core Web UI Components, Page Flow, State Management & API Integration

3.2 การออกแบบสถาปัตยกรรมระบบ

ระบบถูกออกแบบตามสถาปัตยกรรมแบบ Layered Architecture แบ่งออกเป็น 4 ชั้นหลัก ได้แก่ (1) Presentation Layer (Controller) ทำหน้าที่รับคำร้องขอ HTTP และส่งคืนผลลัพธ์ในรูปแบบ JSON (2) Service Layer ทำหน้าที่ประมวลผล Business Logic และควบคุม Transaction (3) Repository Layer ทำหน้าที่เข้าถึงข้อมูลผ่าน Spring Data JPA และ (4) Domain/Entity Layer ที่นิยามโครงสร้างข้อมูลหลักของระบบ โดยแต่ละ Layer ถูกบังคับให้สื่อสารกับ Layer ที่อยู่ติดกันเท่านั้น ห้าม Controller เรียกใช้ Repository โดยตรง นอกจากนี้ยังมีชั้นเสริม ได้แก่ DTO/Mapper สำหรับแยก Entity ออกจากโครงสร้างข้อมูลที่ได้รับส่งผ่าน API, Exception สำหรับจัดการข้อผิดพลาดแบบรวมศูนย์ (Global Exception Handler) และ Config/Security สำหรับการตั้งค่าระบบและการยืนยันตัวตน



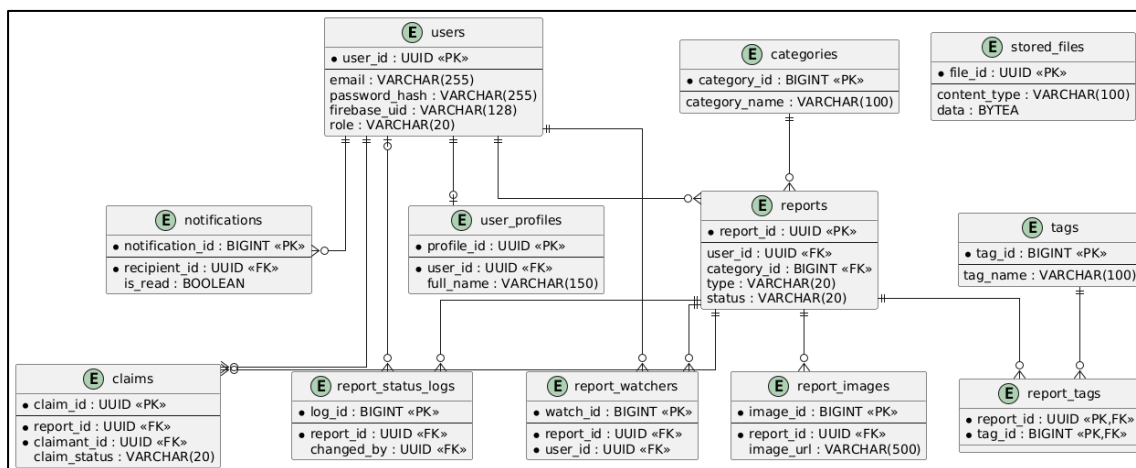
ภาพที่ 1 Component Diagram



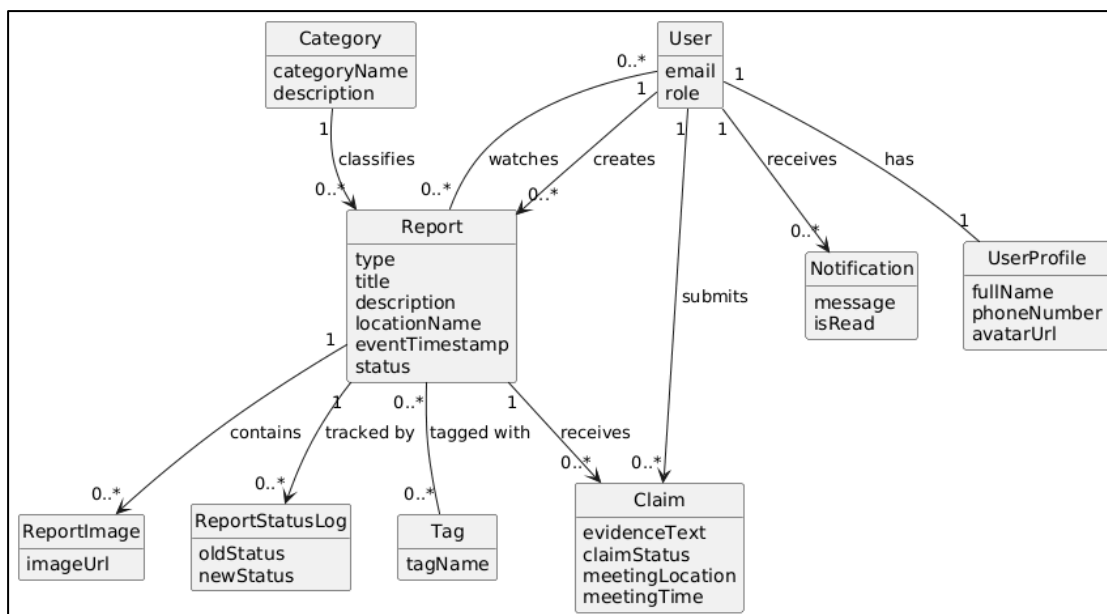
ภาพที่ 2 Deployment Diagram

3.3 การออกแบบฐานข้อมูล

ฐานข้อมูลของระบบประกอบด้วยตารางทั้งหมด 12 ตาราง ออกแบบโดยใช้กลยุทธ์รหัสหลักแบบผสม (Hybrid ID Strategy) กล่าวคือ ตารางที่ผู้ใช้งานเข้าถึงผ่าน URL ได้โดยตรง เช่น users, reports, claims ใช้ชนิดข้อมูล UUID เพื่อป้องกันการคาดเดาลำดับข้อมูล (Enumeration Attack) ในขณะที่ตารางข้อมูลหลักและตารางภายใน เช่น categories, tags, report_images ใช้ชนิดข้อมูล BIGINT เพื่อประสิทธิภาพด้านพื้นที่จัดเก็บและความเร็วในการประมวลผล นอกจากนี้ยังออกแบบคลาสฐาน (Base Class) สองรูปแบบสำหรับการตรวจสอบย้อนกลับ (Audit) ได้แก่ AuditableEntity สำหรับตารางที่แก้ไขข้อมูลได้ (มีทั้ง created_at และ updated_at) และ ImmutableEntity สำหรับตารางประเภทบันทึกประวัติที่ห้ามแก้ไขภายหลัง (มีเฉพาะ created_at) ความสัมพันธ์ระหว่างตารางครอบคลุมทั้งแบบ One-to-One (users – user_profiles) One-to-Many (users – reports, categories – reports, reports – claims เป็นต้น) และ Many-to-Many (reports – tags ผ่านตารางกลาง report_tags) พร้อมทั้งกำหนด Foreign Key Constraint, Index บนคอลัมน์ที่มีการค้นหบ่อย และ Cascade Behavior ที่เหมาะสมกับความหมายทางธุรกิจของแต่ละความสัมพันธ์ เช่น การลบประกาศ (Report) จะลบรูปภาพและประวัติสถานะที่เกี่ยวข้องโดยอัตโนมัติ (ON DELETE CASCADE)



ภาพที่ 3 Entity Relationship (ER) Diagram



ภาพที่ 4 Domain Model (Conceptual Class Diagram)

3.4 การประยุกต์ใช้หลักการ SOLID

การออกแบบคลาสในระบบยึดหลักการ SOLID อย่างเคร่งครัด ดังสรุปในตารางที่ 3 โดยมีรายละเอียดเชิงลึกพร้อมการระบุไฟล์และบรรทัดที่ปรากฏในเอกสาร doc/solid-analysis.md ของโครงการ

ตารางที่ 3 สรุปหลักการ SOLID และตำแหน่งที่ปรากฏในโค้ด

หลักการ	ตัวอย่างคลาส	คำอธิบายโดยสรุป
S (SRP)	ReportStatusChanger, FileStorageService	แต่ละคลาสรับผิดชอบหน้าที่เดียว เช่น ReportStatusChanger จัดการเฉพาะการเปลี่ยนสถานะประกาศ
O (OCP)	ReportState, ClaimEligibilityStrategy	เพิ่มพฤติกรรม/กฎใหม่ได้ด้วยการเพิ่มคลาส ไม่ต้องแก้ไขโค้ดเดิม
L (LSP)	OpenState, MatchPendingState ฯลฯ	คลาสลูกทุกตัวใช้แทนอินเทอร์เฟซ ReportState ได้โดยไม่ทำให้ตรรกะผิดพลาด
I (ISP)	ReportState, Repository แต่ละตัว	อินเทอร์เฟซมีเฉพาะเมธอดที่จำเป็น ไม่รวมเป็นอินเทอร์เฟซขนาดใหญ่
D (DIP)	ClaimServiceImpl, FileUploadController	ใช้ Constructor Injection ผูกกับ Interface เท่านั้น ไม่ผูกกับ Concrete Class

3.5 การประยุกต์ใช้ Design Pattern

ระบบใช้ Enterprise Pattern ครบ 6 รูปแบบตามที่กำหนดในรายวิชา ดังตารางที่ 4 และใช้ GoF Pattern กลุ่ม Behavioral จำนวน 3 รูปแบบ ดังตารางที่ 5

ตารางที่ 4 Enterprise / Architectural Pattern ที่ใช้ในระบบ

Pattern	ตำแหน่งที่ใช้งาน
Layered Architecture	โครงสร้าง package ทั้งโปรเจกต์ (controller → service → repository → domain)
MVC	controller/api (Controller) + Entity/DTO (Model) + JSON Response (View)
Repository Pattern	ทุก interface ใน repository/ ที่ extends JpaRepository
Service Layer Pattern	service/ และ service/impl/ รวม Business Logic
DTO Pattern + Mapper	dto/request, dto/response, mapper/ แยก Entity ออกจาก API Contract
Dependency Injection	Constructor Injection ทุกจุดผ่าน @RequiredArgsConstructor

ตารางที่ 5 GoF Design Pattern และเหตุการณ์การเลือกใช้

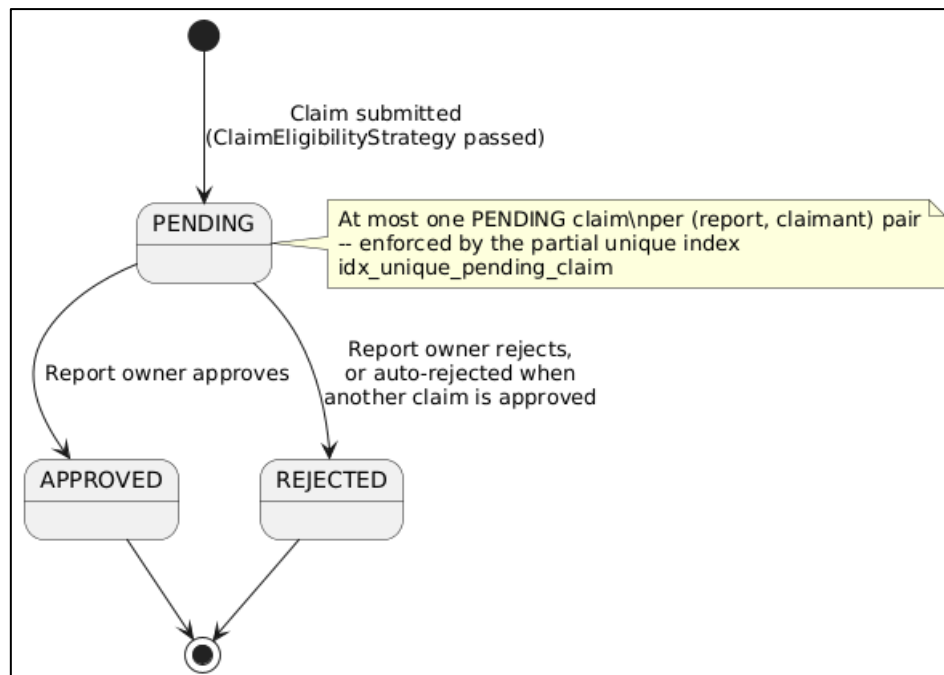
Pattern	ปัญหาที่แก้ไข	คลาสที่เกี่ยวข้อง
State	ควบคุมการเปลี่ยนสถานะของ Report ให้ถูกต้องตาม workflow โดยไม่ใช้ if-else สะสม	ReportStatusChanger, ReportState, OpenState, MatchPendingState, ClaimedState, ClosedState, RejectedState
Observer	แจ้งเตือนผู้ติดตามประกาศโดยอัตโนมัติเมื่อสถานะเปลี่ยนแปลง โดยไม่ผูกติดกับโค้ดแจ้งเตือนโดยตรง	ReportStatusChangedEvent, ReportStatusEventListener, NotificationService
Strategy	ตรวจสอบสิทธิ์การยื่นคำขอเคลมที่มีเงื่อนไขต่างกันตามประเภทประกาศ (LOST/FOUND)	ClaimEligibilityStrategy, LostReportClaimEligibilityStrategy, FoundReportClaimEligibilityStrategy, ClaimEligibilityStrategyResolver

```
stateDiagram-v2
    [*] --> OPEN : Report created
    OPEN --> MATCH_PENDING : A claim is submitted
    MATCH_PENDING --> OPEN : All pending claims are rejected
    MATCH_PENDING --> CLAIMED : Report owner approves a claim
    MATCH_PENDING --> REJECTED : Staff rejects the report (spam / invalid)
    CLAIMED --> CLOSED : Item handover confirmed by owner
    REJECTED --> [*] : Staff rejects the report (spam / invalid)
    CLOSED --> [*]
```

The diagram illustrates the lifecycle of a report, starting from a start state and ending at a final state. The states and transitions are as follows:

- OPEN** (Implemented by `OpenState.java`): The initial state after a report is created.
- MATCH_PENDING** (Implemented by `MatchPendingState.java`): Reached when a claim is submitted from the **OPEN** state.
- CLAIMED** (Implemented by `ClaimedState.java`): Reached when the report owner approves a claim from the **MATCH_PENDING** state.
- CLOSED** (Implemented by `ClosedState.java`): Reached when item handover is confirmed by the owner from the **CLAIMED** state.
- REJECTED** (Implemented by `RejectedState.java`, terminal state): Reached from either **MATCH_PENDING** (if staff rejects the report as spam/invalid) or **OPEN** (if all pending claims are rejected).

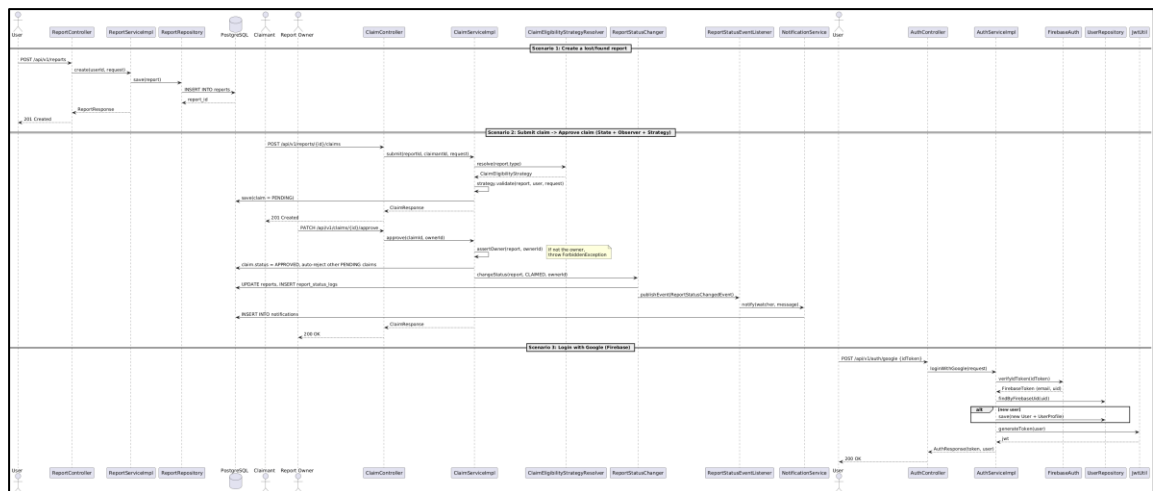
ภาพที่ 6 State Diagram (Report Status)



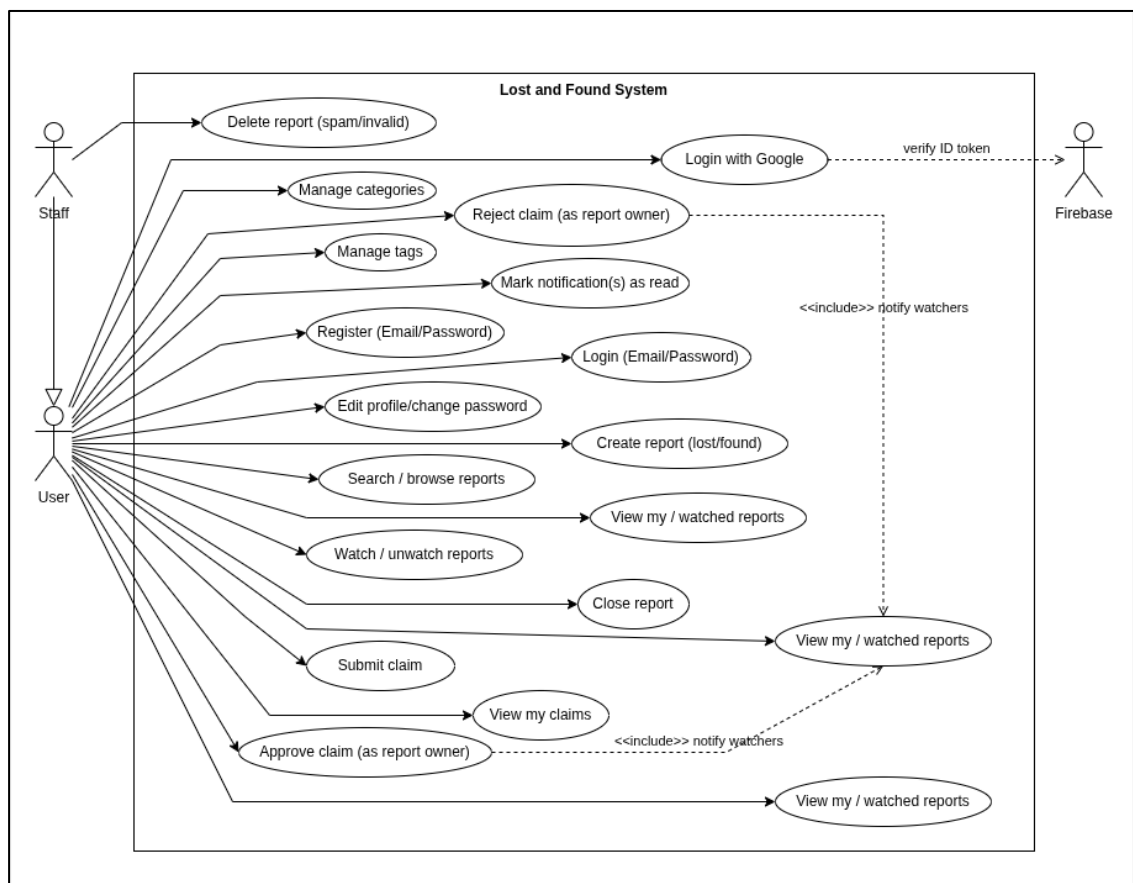
ภาพที่ 7 State Diagram (Claim Status)

3.6 การพัฒนา RESTful API

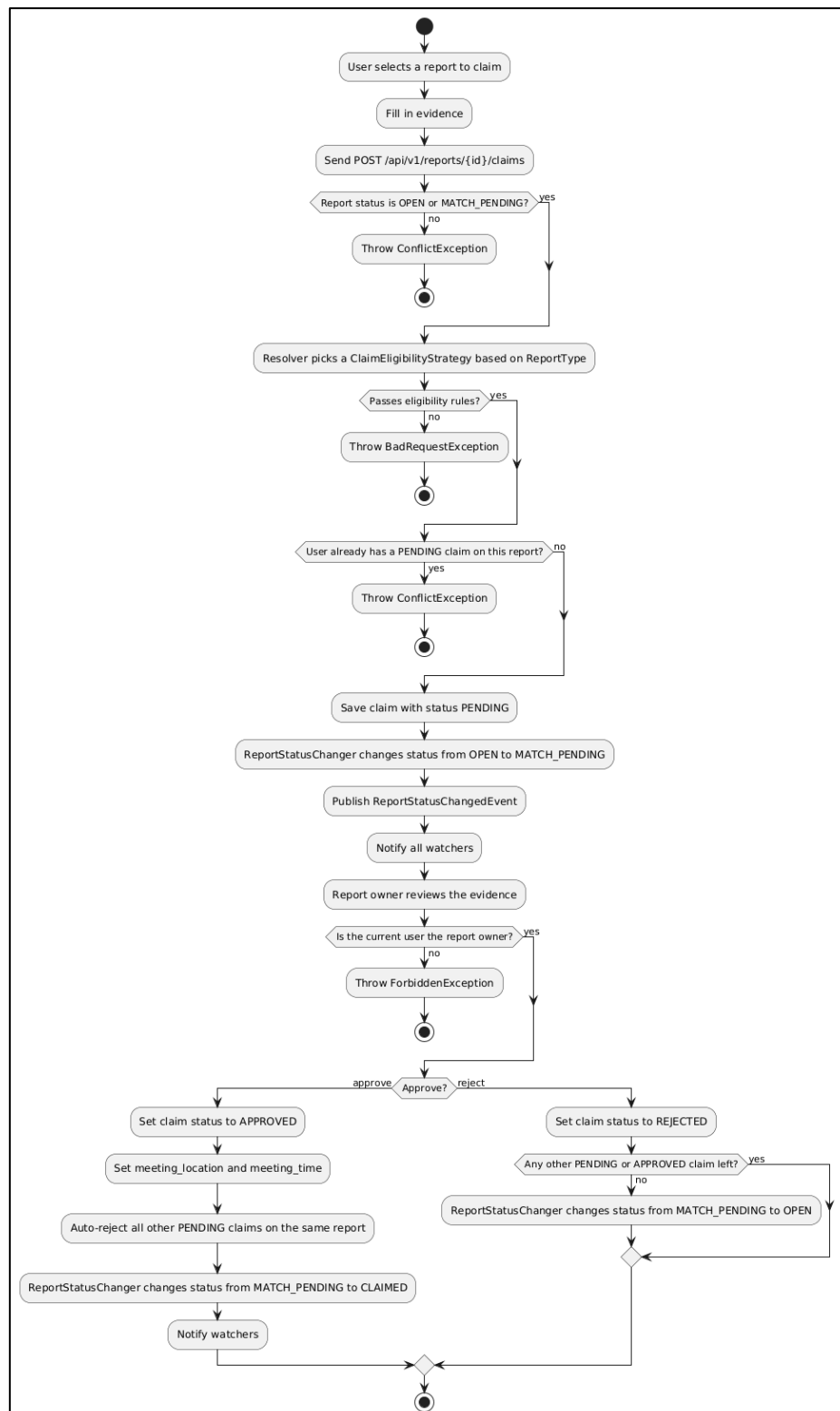
ระบบพัฒนา RESTful API ครอบคลุมทรัพยากรหลัก ได้แก่ Report, Claim, Category, Tag, User, Notification และ File Upload โดยตั้งชื่อ Endpoint แบบอิงทรัพยากร (Resource-based) ใช้ HTTP Method และ Status Code ตามความหมายมาตรฐาน มีการตรวจสอบข้อมูลนำเข้าด้วย Bean Validation (@Valid) มีการจัดการข้อผิดพลาดแบบรวมศูนย์ผ่าน Global Exception Handler (@RestControllerAdvice) และมีเอกสารประกอบ API ผ่าน Swagger UI ที่เข้าถึงได้จริงผ่านเส้นทาง /swagger-ui.html



ภาพที่ 8 Sequence Diagram



ภาพที่ 9 Use Case Diagram



ภาพที่ 10 Activity Diagram

3.7 การบริหารจัดการทีมและ Git Workflow

คณะผู้จัดทำดำเนินงานตามรูปแบบ Git Workflow ที่กำหนดในรายวิชา โดยแต่ละคนมี Branch ส่วนตัวในรูปแบบ ชื่อจริง_รหัสนักศึกษา_section และรวมงานเข้า Branch develop ผ่าน Pull Request ที่ผ่านการตรวจสอบ (Code Review) จากสมาชิกในทีมอย่างน้อยหนึ่งคนก่อน Merge เข้า Branch main ทุกครั้ง การ Commit ข้อความใช้รูปแบบ Conventional Commit (เช่น feat:, fix:, docs:, refactor:) เพื่อให้ประวัติการพัฒนา (Commit History) สามารถตรวจสอบย้อนหลังได้ว่าแต่ละการเปลี่ยนแปลงมีวัตถุประสงค์ใด

3.8 การทดสอบและการนำระบบขึ้นใช้งานจริง (Deployment)

การทดสอบระบบดำเนินการตามแนวคิดพีระมิดการทดสอบ (Cohn, 2009; Fowler, 2012b) โดยแบ่งเป็น 6 ส่วน ดังนี้

- Unit Test ใช้ JUnit 5 ร่วมกับ Mockito ทดสอบ Business Logic ในชั้น Service, State Pattern และ Strategy Pattern โดยจำลอง Repository รันด้วยคำสั่ง mvn test ตัวอย่างที่เน้นเป็นพิเศษคือ ClaimServiceImpl ซึ่งมีกฎธุรกิจซับซ้อนที่สุดของระบบ
- Integration Test ใช้ Spring Boot Test ร่วมกับ Testcontainers เพื่อทดสอบ Flow ของการเคลมผ่านทุกชั้นจนถึงฐานข้อมูล PostgreSQL จริง (ClaimFlowIntegrationTest) ชุดทดสอบนี้ไม่ถูกรวมในการรัน mvn test ปกติ และต้องรันแยกด้วยคำสั่ง mvn test -Dtest=ClaimFlowIntegrationTest -DexcludesIntegrationTests=
- API Test ทดสอบ RESTful API ผ่าน Swagger UI ครอบคลุม 22 Endpoint ทั้งกรณีปกติและกรณีผิดพลาด พร้อมบันทึก HTTP Status Code ที่ได้จริง
- Authentication/Authorization Test ทดสอบการเข้าสู่ระบบ การเรียก API โดยไม่มีหรือมี Token และการเรียกโดยไม่มีสิทธิ์ (401/403)
- Frontend Functional Test ทดสอบการใช้งานหน้าเว็บบนระบบที่ Deploy จริง รวมถึงทดสอบบนโทรศัพท์มือถือ Android
- Code Coverage วัดด้วย JaCoCo เวอร์ชัน 0.8.12 ผ่าน Maven

สภาพแวดล้อมการทดสอบใช้ Java 17 และ Spring Boot โดย Integration Test ใช้ PostgreSQL ผ่าน Testcontainers ส่วนระบบที่ Deploy จริงเชื่อมต่อ PostgreSQL บน Neon Cloud และยืนยันตัวตนผ่าน Firebase

สำหรับการนำระบบขึ้นใช้งานจริง ระบบถูกบรรจุ (Containerize) ด้วย Docker โดยมี Dockerfile แยกสำหรับ Backend และ Frontend และใช้ Docker Compose รันทั้งระบบพร้อมกันในสภาพแวดล้อมสำหรับพัฒนา (Local Development) ส่วนการนำระบบขึ้น Deploy บน Render โดยเชื่อมต่อฐานข้อมูล PostgreSQL บน Neon Cloud

บทที่ 4

ผลการดำเนินการ

4.1 ผลการออกแบบฐานข้อมูล

ผลการออกแบบฐานข้อมูลได้ตารางทั้งหมด 12 ตาราง ดังตารางที่ 6 ซึ่งครอบคลุมความสัมพันธ์ครบทั้งสามรูปแบบตามที่กำหนดในขอบเขตของรายวิชา พร้อมทั้งมีการกำหนด Index บนคอลัมน์ที่มีการค้นหาบ่อย เช่น reports.status, reports.category_id และมี Partial Unique Index เพื่อป้องกันการยื่นคำขอเคลมซ้ำซ้อนในสถานะ PENDING บนประกาศเดียวกัน

ตารางที่ 6 รายชื่อตารางในฐานข้อมูลทั้งหมด

ลำดับ	ชื่อตาราง	ชนิด Primary Key	คำอธิบาย
1	users	UUID	บัญชีผู้ใช้งานและข้อมูลการเข้าสู่ระบบ
2	user_profiles	UUID	ข้อมูลส่วนตัวของผู้ใช้ (ความสัมพันธ์ 1:1 กับ users)
3	categories	BIGINT	หมวดหมู่ของประกาศ
4	tags	BIGINT	แท็กของประกาศ
5	reports	UUID	ประกาศแจ้งของหาย/พบของ
6	report_images	BIGINT	รูปภาพประกอบประกาศ
7	report_status_logs	BIGINT	ประวัติการเปลี่ยนสถานะของประกาศ
8	report_watchers	BIGINT	ผู้ติดตามประกาศ
9	claims	UUID	คำขอเคลมของประกาศ
10	report_tags	Composite	ตารางกลางความสัมพันธ์ Many-to-Many ระหว่าง reports และ tags
11	notifications	BIGINT	ข้อความแจ้งเตือนผู้ใช้งาน
12	stored_files	UUID	ไฟล์รูปภาพที่อัปโหลดเข้าสู่ระบบ

4.2 ผลการพัฒนา RESTful API

ระบบพัฒนา RESTful API ที่รองรับการทำงานแบบ CRUD ครบถ้วนสำหรับทรัพยากรหลักอย่างน้อยสองรายการตามที่กำหนด พร้อมทั้งมี Pagination และ Sorting สำหรับ

Endpoint การค้นหาประกาศ ตัวอย่าง Endpoint หลักแสดงในตารางที่ 7 (รายการ Endpoint ทั้งหมดแสดงไว้ในภาคผนวก ก)

ตารางที่ 7 ตัวอย่าง RESTful API Endpoint หลัก

HTTP Method	Endpoint	คำอธิบาย
POST	/api/v1/reports	สร้างประกาศแจ้งของหาย/พบของใหม่
GET	/api/v1/reports	ค้นหาประกาศ พร้อม Pagination และ Sorting
POST	/api/v1/reports/{id}/claims	ยื่นคำขอเคลมสำหรับประกาศที่ระบุ
PATCH	/api/v1/claims/{id}/approve	อนุมัติคำขอเคลม (เฉพาะเจ้าของประกาศ)
PATCH	/api/v1/claims/{id}/reject	ปฏิเสธคำขอเคลม (เฉพาะเจ้าของประกาศ)
DELETE	/api/v1/admin/reports/{id}	ลบประกาศที่ไม่เหมาะสม (เฉพาะ Staff)
GET	/api/v1/notifications	ดูรายการแจ้งเตือนของผู้ใช้งาน

ระบบรองรับการจัดการข้อผิดพลาดผ่าน Global Exception Handler โดยส่งคืน HTTP Status Code ที่เหมาะสมตามสถานการณ์ เช่น 400 (Bad Request) เมื่อข้อมูลไม่ผ่านการตรวจสอบ 403 (Forbidden) เมื่อผู้ใช้ไม่มีสิทธิ์ดำเนินการ 404 (Not Found) เมื่อไม่พบทรัพยากร และ 409 (Conflict) เมื่อเกิดความขัดแย้งของข้อมูล เช่น การยื่นคำขอเคลมซ้ำซ้อน เอกสารประกอบ API ทั้งหมดสามารถเข้าถึงได้ผ่าน Swagger UI

4.3 ผลการจัดทำแผนภาพระบบ (Diagrams)

โครงการได้จัดทำแผนภาพประกอบการออกแบบระบบครบถ้วนตามที่กำหนด ได้แก่ Use Case Diagram พร้อม Use Case Description, Domain Model, Class Diagram ที่ระบุตำแหน่ง Design Pattern, ER Diagram, Sequence Diagram อย่างน้อย 3 สถานการณ์หลัก, Activity Diagram, Component Diagram, Deployment Diagram และ State Diagram สำหรับ Entity ที่มีสถานะ โดยแผนภาพทั้งหมด จัดเก็บไว้ในโฟลเดอร์ diagrams ของ Repository

สำหรับ State Diagram ได้จัดทำแยกเป็นสองส่วนตามจำนวน Entity ที่มีสถานะในระบบ ได้แก่ สถานะของ Report (OPEN, MATCH_PENDING, CLAIMED, CLOSED, REJECTED) และ สถานะของ Claim (PENDING, APPROVED, REJECTED) โดยเส้นทางการเปลี่ยนสถานะทั้งหมด สอดคล้องโดยตรงกับการ implement ด้วย State Pattern ในแพ็คเกจ service/state/ กล่าวคือแต่ละสถานะของ Report ถูกห่อหุ้มไว้ในคลาสของตนเอง (OpenState, MatchPendingState, ClaimedState, ClosedState, RejectedState) ซึ่งทำหน้าที่ตรวจสอบว่าสามารถเปลี่ยนไปยังสถานะถัดไปได้หรือไม่ ก่อนที่ ReportStatusChanger จะดำเนินการเปลี่ยนสถานะจริงและบันทึกประวัติลงตาราง report_status_logs ทุกครั้ง ส่วนสถานะของ Claim ถูกควบคุมด้วยเงื่อนไขทางธุรกิจร่วมกับ Partial Unique Index (idx_unique_pending_claim) ที่ระดับฐานข้อมูล เพื่อป้องกันผู้ใช้ ยื่นคำขอเคลมสถานะ PENDING ซ้ำซ้อนในประกาศเดียวกัน

4.4 ผลการทดสอบระบบ

การทดสอบระบบ ผู้รับผิดชอบงานทดสอบ (QA) ผลสรุปตามตารางที่ 8 พบว่าผ่านทุกกรณี ไม่พบกรณีที่ล้มเหลว รายงานผลการทดสอบฉบับเต็มจัดเก็บไว้ที่ test/test-report.md ของ Repository

ตารางที่ 8 สรุปผลการทดสอบระบบ

ประเภทการทดสอบ	เครื่องมือ/วิธีการ	จำนวนกรณีทดสอบ	ผ่าน	ไม่ผ่าน
Automated Test ชุดหลัก (Unit Test)	JUnit 5 + Mockito (mvn test)	116	116	0
Integration Test	Spring Boot Test + Testcontainers	5	5	0
API Test	Swagger UI	22	22	0
Authentication/Authorization Test	Swagger UI	5	5	0
Frontend Functional Test	ทดสอบบนเว็บที่ Deploy	6	6	0

ผลการรันชุดทดสอบอัตโนมัติด้วยคำสั่ง mvn test พบว่ารันทั้งหมด 116 กรณี ไม่มี Failure, Error หรือ Skipped และ Build สำเร็จ ใช้เวลาประมาณ 22.5 วินาที ดังภาพที่ 11

```

PS C:\Users\LOQ\Documents\CP353002-Lost-and-Found\code> mvn test
xample.lostandfound.service.ReportStatusChangerTest
[INFO] Running com.example.lostandfound.service.strategy.ClaimEligibilityStrategyTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s -- in com.e
xample.lostandfound.service.strategy.ClaimEligibilityStrategyTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 116, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ CP353002_Group12_LostAndFound ---
[INFO] Loading execution data file C:\Users\LOQ\Documents\CP353002-Lost-and-Found\tar
get\jacoco.exec
[INFO] Analyzed bundle 'CP353002_Group12_LostAndFound' with 61 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 22.538 s
[INFO] Finished at: 2569-10-07T14:48:25+07:00
[INFO] -----
PS C:\Users\LOQ\Documents\CP353002-Lost-and-Found\code>

```

ภาพที่ 11 ผลการรันชุดทดสอบอัตโนมัติด้วยคำสั่ง mvn test

JaCoCo วิเคราะห์โค้ดทั้งหมด 61 คลาส ได้รับความครอบคลุมดังตารางที่ 9 โดย Class Coverage อยู่ที่ร้อยละ 82 (ประมาณ 50 จาก 61 คลาสถูกรันอย่างน้อยหนึ่งครั้ง) ส่วน Instruction, Line และ Method Coverage อยู่ในช่วงร้อยละ 61–63 และ Branch Coverage ต่ำสุดที่ร้อยละ 60 สะท้อนว่ายังมีบางกิ่งของเงื่อนไขที่ชุดทดสอบไม่ได้ครอบคลุม อย่างไรก็ตาม ตามที่ Fowler (2012a) ชี้ว่าตัวเลข Coverage เพียงอย่างเดียวไม่ใช่ตัวชี้วัดคุณภาพของการทดสอบ ทีมจึงให้น้ำหนักกับการออกแบบกรณีทดสอบสำหรับกฎธุรกิจสำคัญ ดังรายละเอียดในตารางที่ 10

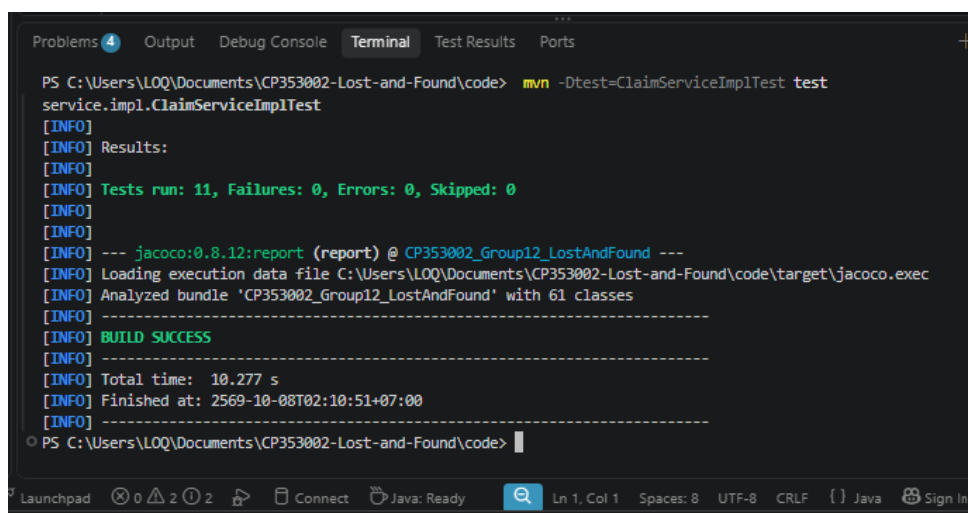
ตารางที่ 9 ผลการวัดความครอบคลุมของการทดสอบ (Code Coverage)

ตัวชี้วัด	ความครอบคลุม
Instruction Coverage	63%
Branch Coverage	60%
Line Coverage	62%
Method Coverage	61%
Class Coverage	82%

ชุดทดสอบ ClaimServiceImplTest ทดสอบกฎธุรกิจของการยืนยันและตัดสินใจขอเคลมรวม 11 กรณี ผ่านทั้งหมด (ภาพที่ 11) ครอบคลุมการเปลี่ยนสถานะตาม State Pattern (กรณีที่ 1, 2, 6, 9, 10) กฎการตรวจสอบสิทธิ์ยื่นเคลมตาม Strategy Pattern (กรณีที่ 3, 4) การป้องกันการยื่นซ้ำ (กรณีที่ 5) และการตรวจสอบว่าผู้ตัดสินเป็นเจ้าของประกาศจริง (กรณีที่ 7, 11)

ตารางที่ 10 ผลการทดสอบ Unit Test ของ ClaimServiceImpl

ลำดับ	กรณีทดสอบ	ผลลัพธ์ที่คาดหวัง	ผล
1	ยื่นเคลมประกาศที่เปิดอยู่ (คำขอแรก)	สร้าง Claim สำเร็จ และ Report เปลี่ยนเป็น MATCH_PENDING	ผ่าน
2	ยื่นเคลมเมื่อ Report เป็น MATCH_PENDING อยู่แล้ว	สร้าง Claim สำเร็จ แต่ไม่เปลี่ยนสถานะ Report ซ้ำ	ผ่าน
3	ยื่นเคลมประกาศที่ปิดแล้ว	BadRequestException	ผ่าน
4	ยื่นเคลมประกาศของตนเอง	BadRequestException	ผ่าน
5	ยื่นเคลมซ้ำขณะยังมีคำขอ PENDING	ConflictException	ผ่าน
6	อนุมัติคำขอเคลม	Claim เป็น APPROVED, Claim อื่นเป็น REJECTED และ Report เป็น CLAIMED	ผ่าน
7	อนุมัติโดยผู้ที่ไม่ใช่เจ้าของประกาศ	ForbiddenException	ผ่าน
8	อนุมัติคำขอที่ตัดสินใจไปแล้ว	BadRequestException	ผ่าน
9	ปฏิเสธคำขอเมื่อไม่มีคำขอ PENDING/APPROVED เหลือ	Claim เป็น REJECTED และ Report กลับเป็น OPEN	ผ่าน
10	ปฏิเสธคำขอขณะที่ยังมีคำขออื่นเหลืออยู่	Claim เป็น REJECTED แต่ Report ยังเป็น MATCH_PENDING	ผ่าน
11	ปฏิเสธโดยผู้ที่ไม่ใช่เจ้าของประกาศ	ForbiddenException	ผ่าน



```

PS C:\Users\LOQ\Documents\CP353002-Lost-and-Found\code> mvn -Dtest=ClaimServiceImplTest test
service.impl.ClaimServiceImplTest
[INFO] Results:
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ CP353002_Group12_LostAndFound ---
[INFO] Loading execution data file C:\Users\LOQ\Documents\CP353002-Lost-and-Found\code\target\jacoco.exec
[INFO] Analyzed bundle 'CP353002_Group12_LostAndFound' with 61 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.277 s
[INFO] Finished at: 2569-10-08T02:10:51+07:00
[INFO] -----
PS C:\Users\LOQ\Documents\CP353002-Lost-and-Found\code>

```

ภาพที่ 12 ผลการรัน Unit Test ของ ClaimServiceImpl

Integration Test(ClaimFlowIntegrationTest) รันกับ PostgreSQL ผ่าน Testcontainers ทั้งหมด 5 กรณี ผ่านทั้งหมด ใช้เวลาประมาณ 1 นาที 4 วินาที ครอบคลุมสถานการณ์หลักตามตารางที่ 11 (ภาพที่ 13)

ตารางที่ 11 ผลการทดสอบ Integration Test

สถานการณ์ที่ทดสอบ	ผลลัพธ์ที่คาดหวัง	ผล
สร้างประกาศ	สร้างประกาศสำเร็จ	ผ่าน
ยื่นคำขอเคลม	สร้าง Claim สำเร็จ	ผ่าน
เรียกดูคำขอเคลม	คืนข้อมูล Claim ที่ถูกต้อง	ผ่าน
Flow การเคลมที่ไม่ถูกต้อง	ระบบปฏิเสธคำขอ	ผ่าน

```

Problems 8 Output Debug Console Terminal Test Results Ports
2026-10-08T17:40:38.744+07:00 WARN 28176 --- [CP353002_Group12_LostAndFound] [ionShutdownHook] com.zaxxer.hikari.pool.PoolBase
action has been closed.). Possibly consider using a shorter maxLifetime value.
2026-10-08T17:40:38.744+07:00 WARN 28176 --- [CP353002_Group12_LostAndFound] [ionShutdownHook] com.zaxxer.hikari.pool.PoolBase
action has been closed.). Possibly consider using a shorter maxLifetime value.
2026-10-08T17:40:38.744+07:00 WARN 28176 --- [CP353002_Group12_LostAndFound] [ionShutdownHook] com.zaxxer.hikari.pool.PoolBase
action has been closed.). Possibly consider using a shorter maxLifetime value.
2026-10-08T17:40:38.746+07:00 WARN 28176 --- [CP353002_Group12_LostAndFound] [ionShutdownHook] com.zaxxer.hikari.pool.PoolBase
action has been closed.). Possibly consider using a shorter maxLifetime value.
[ERROR] Surefire is going to kill self fork JVM. The exit has elapsed 30 seconds after System.exit(0).
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ CP353002_Group12_LostAndFound ---
[INFO] Loading execution data file C:\Users\LOQ\Documents\CP353002-Lost-and-Found\code\target\jacoco.exec
[INFO] Analyzed bundle 'CP353002_Group12_LostAndFound' with 61 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 01:04 min
[INFO] Finished at: 2569-10-08T17:41:09+07:00
[INFO]
PS C:\Users\LOQ\Documents\CP353002-Lost-and-Found\code> mvn test -Dtest=ClaimFlowIntegrationTest -DexcludesIntegrationTests=

```

ภาพที่ 13 ผลการรัน Integration Test (ClaimFlowIntegrationTest)

การทดสอบ API ผ่าน Swagger UI ครอบคลุม 22 Endpoint ผ่านทั้งหมด และได้ HTTP Status Code ตรงตามที่ต้องการ (200, 201 และ 204) สรุปตามกลุ่มทรัพยากรได้ดังตารางที่ 12 รายละเอียดทั้ง 22 กรณีอยู่ในภาคผนวก ข ตัวอย่างผลการทดสอบแสดงในภาพที่ 13

ตารางที่ 12 สรุปผลการทดสอบ API แยกตามกลุ่มทรัพยากร

กลุ่มทรัพยากร	Endpoint ที่ทดสอบ	จำนวน	ผล
Authentication	POST /auth/register, /auth/login, /auth/google	3	ผ่านทั้งหมด
User	GET, PUT /users/me	2	ผ่านทั้งหมด
Report	POST /reports, GET /reports, GET /reports/{id}	3	ผ่านทั้งหมด

กลุ่มทรัพยากร	Endpoint ที่ทดสอบ	จำนวน	ผล
Watch	POST, DELETE /reports/{id}/watch	2	ผ่านทั้งหมด
Claim	POST /reports/{id}/claims, GET /claims/me, PATCH /claims/{id}/approve	3	ผ่านทั้งหมด
Admin	DELETE /admin/reports/{id}	1	ผ่านทั้งหมด
Category/Tag	GET, POST /categories และ GET, POST /tags	4	ผ่านทั้งหมด
Notification	GET /notifications, PATCH /notifications/{id}/read	2	ผ่านทั้งหมด
File	POST /uploads, GET /files/{id}	2	ผ่านทั้งหมด
รวม		22	ผ่านทั้งหมด
กลุ่มทรัพยากร	Endpoint ที่ทดสอบ	จำนวน	ผล

POST : /api/v1/reports



ภาพที่ 14 ตัวอย่างผลการทดสอบ API ผ่าน Swagger UI

ผลการทดสอบการยืนยันตัวตนและสิทธิ์การเข้าถึงเป็นไปตามตารางที่ 13 โดยการตอบกลับ 401 และ 403 สอดคล้องกับ RestAuthenticationEntryPoint และ RestAccessDeniedHandler ที่กำหนดไว้ในชั้น Security

ตารางที่ 13 ผลการทดสอบการยืนยันตัวตนและสิทธิ์การเข้าถึง

กรณีทดสอบ	ผลลัพธ์ที่คาดหวัง	ผล
เข้าสู่ระบบด้วยข้อมูลที่ถูกต้อง	เข้าสู่ระบบสำเร็จ	ผ่าน
เข้าสู่ระบบด้วยข้อมูลที่ไม่ถูกต้อง	ระบบปฏิเสธการยืนยันตัวตน	ผ่าน
เรียก API ที่ต้องยืนยันตัวตนโดยไม่มี Token	401 Unauthorized	ผ่าน
เรียก API ด้วย Token ที่ถูกต้อง	ระบบรับคำขอ	ผ่าน

กรณีทดสอบ	ผลลัพธ์ที่คาดหวัง	ผล
เรียก API โดยไม่มีสิทธิ์เพียงพอ	403 Forbidden	ผ่าน

การทดสอบฟังก์ชันของ Frontend บนระบบที่ Deploy จริง ผ่านทั้ง 6 กรณีตามตารางที่ 14

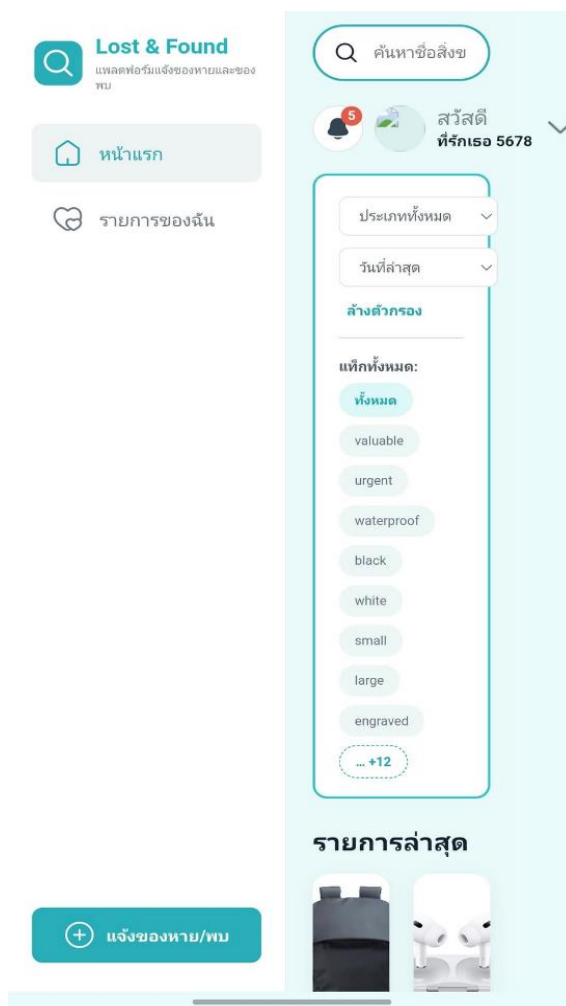
ตารางที่ 14 ผลการทดสอบฟังก์ชันของ Frontend

กรณีทดสอบ	ผลลัพธ์ที่คาดหวัง	ผล
ผู้ใช้เข้าสู่ระบบ	แสดงหน้า Dashboard	ผ่าน
ค้นหาของหาย/ของที่พบ	แสดงผลที่ตรงกับคำค้น	ผ่าน
สร้างประกาศของหาย	สร้างประกาศสำเร็จ	ผ่าน
สร้างประกาศของที่พบ	สร้างประกาศสำเร็จ	ผ่าน
ดูรายการของฉัน	แสดงรายการของผู้ใช้	ผ่าน
แก้ไขข้อมูลบัญชี	ข้อมูลถูกอัปเดต	ผ่าน

จากการทดสอบพบข้อบกพร่อง 1 รายการ ระดับความรุนแรงต่ำ เป็นปัญหาการแสดงผลหน้าแรกบนโทรศัพท์มือถือ Android กล่าวคือเมนูด้านซ้ายและเนื้อหาหลักแสดงอยู่ข้างกัน ทำให้พื้นที่เนื้อหาด้านขวามีขนาดแคบและไม่เหมาะกับหน้าจอขนาดเล็ก ข้อบกพร่องนี้ได้รับการแก้ไขแล้ว ดังตารางที่ 15 และภาพที่ 15

ตารางที่ 15 รายงานข้อบกพร่องที่พบจากการทดสอบ

รหัส	ปัญหาที่พบ	ส่วนที่พบ	ความรุนแรง	สถานะ
BUG-01	หน้าแรกบนมือถือแสดงเมนูข้างและเนื้อหาหลักเคียงกัน ทำให้พื้นที่เนื้อหาแคบ	หน้าแรก (Android)	ต่ำ	แก้ไขแล้ว



ภาพที่ 15 ภาพหน้าจอข้อบกพร่อง BUG-01 บนโทรศัพท์มือถือ Android

4.5 ผลการนำระบบขึ้นใช้งานจริง

ระบบถูกนำขึ้นใช้งานจริงบน Render ผ่าน Docker Container โดยสามารถเข้าถึงส่วนติดต่อผู้ใช้งาน (Frontend) และเอกสาร API (Swagger UI) ผ่าน URL สาธารณะได้จริง ฐานข้อมูลเชื่อมต่อกับ PostgreSQL บน Neon Cloud ทั้งนี้เนื่องจากข้อจำกัดของแผนการใช้งานแบบไม่เสียค่าใช้จ่าย (Free Tier) บริการฝั่ง Backend อาจเข้าสู่สถานะพักการทำงาน (Sleep) เมื่อไม่มีการใช้งานเป็นระยะเวลาหนึ่ง และต้องใช้เวลาประมาณ 50 วินาทีขึ้นไปในการเริ่มทำงานใหม่เมื่อมีการเรียกใช้งานครั้งแรก

บทที่ 5

สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผลการดำเนินโครงการ

โครงการพัฒนาระบบ Lost and Found สามารถดำเนินการได้ครบถ้วนตามวัตถุประสงค์และขอบเขตที่กำหนด โดยได้ระบบที่ทำงานได้จริงตั้งแต่การแจ้งของหาย/พบของการค้นหา การติดตามประกาศ การยื่นและอนุมัติคำขอเคลม การแจ้งเตือนอัตโนมัติ ไปจนถึงการจัดการข้อมูลโดยผู้ดูแลระบบ ระบบถูกออกแบบตามสถาปัตยกรรมแบบ Layered Architecture ประยุกต์ใช้หลักการ SOLID และ Design Pattern กลุ่ม Behavioral จำนวน 3 รูปแบบอย่างมีเหตุผลรองรับ ฐานข้อมูลถูกออกแบบอย่างเป็นระบบด้วยกลยุทธ์ Hybrid ID และมีความสัมพันธ์ครบทั้งสามรูปแบบ อีกทั้งยังสามารถนำระบบขึ้นใช้งานจริงบน Cloud ได้สำเร็จ

ในด้านการทดสอบ ชุดทดสอบอัตโนมัติ 116 กรณี Integration Test 5 กรณี API Test 22 กรณี การทดสอบการยืนยันตัวตน 5 กรณี และการทดสอบฟังก์ชัน Frontend 6 กรณี ผ่านทั้งหมด โดยมีความครอบคลุมของโค้ดระดับ Line ร้อยละ 62

5.2 อภิปรายผล

ระหว่างการพัฒนา ระบบ คณะผู้จัดทำพบประเด็นสำคัญที่ควรอภิปรายหลายประการ ประการแรก การเลือกใช้กลยุทธ์รหัสหลักแบบผสม (Hybrid ID Strategy) ส่งผลให้ต้องออกแบบคลาสฐานสองรูปแบบ ได้แก่ AuditableEntity และ ImmutableEntity แทนการใช้คลาสฐานเดียวสำหรับ Entity ทุกประเภท แม้เพิ่มความซับซ้อนบ้าง แต่ช่วยให้ระบบยืดหยุ่นและสอดคล้องกับความหมายทางธุรกิจของแต่ละตารางมากขึ้น ประการที่สอง การกำหนดสิทธิ์อนุมัติคำขอเคลมให้เป็นของเจ้าของประกาศแทนผู้ดูแลระบบ สะท้อนว่าการออกแบบสิทธิ์การเข้าถึงควรยึดตรรกะทางธุรกิจจริง มากกว่ายึดบทบาทองค์กรเพียงอย่างเดียว และประการที่สาม การใช้ดัชนีเฉพาะส่วน (Partial Unique Index) แสดงว่าข้อจำกัดทางธุรกิจบางอย่างไม่สามารถกำหนดผ่าน Annotation ของ JPA/Hibernate ได้ จำเป็นต้องอาศัย schema.sql เป็นแหล่งความจริงเพียงแหล่งเดียวของโครงสร้างฐานข้อมูล

ประเด็นเรื่องการแยกคลาสฐาน AuditableEntity และ ImmutableEntity สมควรอภิปรายเพิ่มเติม เนื่องจากส่งผลต่อโครงสร้างโค้ดทั้งระบบ ในช่วงแรกทีมพิจารณาใช้คลาสฐานเดียว (BaseEntity) ที่มีทั้ง createdAt และ updatedAt สำหรับทุก Entity เพื่อความเรียบง่าย แต่เมื่อวิเคราะห์พบว่าตารางแบ่งได้เป็นสองกลุ่ม กลุ่มแรกคือข้อมูลที่แก้ไขได้ภายหลัง เช่น โปรไฟล์ผู้ใช้ หรือ รายละเอียดประกาศ ซึ่งต้องมี updated_at ส่วนกลุ่มที่สองคือข้อมูลบันทึกประวัติ เช่น ประวัติการ

เปลี่ยนสถานะหรือข้อความแจ้งเตือน ซึ่งตามหลัก Auditability ไม่ควรถูกแก้ไขหลังสร้างขึ้น หากใช้ BaseEntity ที่มี updatedAt เสมอ จะเปิดช่องให้แก้ไขข้อมูลประวัติย้อนหลังได้ ซึ่งลดความน่าเชื่อถือของข้อมูลในการตรวจสอบข้อพิพาท ทีมจึงแยกคลาสฐานตามหลัก Single Responsibility Principle และ Interface Segregation Principle ใน SOLID โดย AuditableEntity รับผิดชอบข้อมูลที่เปลี่ยนแปลงได้ ส่วน ImmutableEntity รับผิดชอบข้อมูลบันทึกประวัติที่ไม่เปลี่ยนแปลง แนวทางนี้ยังสอดคล้องกับกลยุทธ์ Hybrid ID เนื่องจากไม่กำหนด @Id ในคลาสฐาน แต่ให้ Entity ลูกเลือกชนิด Primary Key เอง (UUID หรือ BIGINT) ข้อดีคือ Entity ประวัติจะไม่มี setter หรือ updatedAt ตั้งแต่ระดับคอมไพล์ ป้องกันการแก้ไขโดยไม่ตั้งใจ นักพัฒนาทราบเจตนาการออกแบบได้ทันทีจากคลาสที่ extends และโครงสร้าง Entity กับตารางตรงกัน เนื่องจากตารางเหล่านี้ไม่มีคอลัมน์ updated_at ตั้งแต่ระดับ schema.sql ลดความเสี่ยงข้อผิดพลาดเมื่อตรวจสอบ Schema ด้วย Hibernate ในโหมด ddl-auto=validate

ผลการทดสอบสะท้อนประเด็นที่น่าสนใจหลายประการ ประการแรก การออกแบบให้ Service ขึ้นกับ Interface ร่วมกับการแยกกฎธุรกิจเป็น State และ Strategy ทำให้ ClaimServiceImpl ถูกทดสอบได้ครบ 11 กรณีด้วย Mockito โดยไม่ต้องต่อฐานข้อมูลจริง ซึ่งเป็นผลที่ได้เห็นได้ชัดของหลัก SOLID ประการที่สอง Integration Test กับ PostgreSQL จริงผ่าน Testcontainers ยืนยันพฤติกรรมระดับฐานข้อมูลที่ Mock ตรวจสอบไม่ได้ เช่น Constraint และ Partial Unique Index ประการที่สาม ความครอบคลุมระดับ Line ร้อยละ 62 และ Branch ร้อยละ 60 ยังเป็นระดับปานกลาง ตามที่ Fowler (2012a) ชี้ ตัวเลขนี้ควรใช้ค้นหาส่วนที่ยังไม่ถูกทดสอบมากกว่าใช้เป็นเป้าหมายในตัวเอง และประการที่สี่ ข้อบกพร่อง BUG-01 ถูกพบจากการทดสอบด้วยมือบนโทรศัพท์จริง ไม่ใช่ชุดทดสอบอัตโนมัติ ซึ่งสอดคล้องกับแนวคิดพิระมิดการทดสอบ (Fowler, 2012b) ที่ว่าแต่ละชั้นการทดสอบตรวจพบข้อผิดพลาดคนละประเภท

5.3 ข้อเสนอแนะ

1. ควรเพิ่มกรณีทดสอบเพื่อยกระดับ Branch Coverage โดยเฉพาะเส้นทางที่เกิดข้อผิดพลาด และควรวัด Coverage แยกตามแพ็คเกจเพื่อดูว่าส่วนใดยังขาด
2. ควรเปลี่ยน API Test จากการทดสอบด้วยมือผ่าน Swagger เป็นการทดสอบอัตโนมัติ และเพิ่มกรณีของ Endpoint ที่ยังไม่ได้ทดสอบ ได้แก่ การปิดประกาศและการปฏิเสธคำขอเคลม
3. ควรขยาย Integration Test ให้ครอบคลุม Flow อื่น เช่น การติดตามประกาศและการสร้างการแจ้งเตือนผ่าน Observer Pattern
4. ควรเพิ่มการทดสอบ UI อัตโนมัติในจำนวนน้อยสำหรับ Flow หลัก และทดสอบ Responsive Design บนอุปกรณ์หลายขนาด

5. ควรพิจารณาใช้บริการ Cloud ระดับที่สูงกว่า Free Tier เพื่อแก้ปัญหาการพักการทำงาน
ของ Backend และการสูญหายของไฟล์ที่อัปโหลด
6. ควรพัฒนาการค้นหาขั้นสูง เช่น การค้นหาตามตำแหน่งที่ตั้ง (Location-based Search)
เพื่อเพิ่มประสิทธิภาพการจับคู่ประกาศของหายกับของที่พบ

บรรณานุกรม

- Cohn, M. (2009). Succeeding with Agile: Software development using Scrum. Addison-Wesley.
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation). University of California, Irvine.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.
- JaCoCo. (n.d.). JaCoCo Java code coverage library. <https://www.jacoco.org/jacoco/>
- JUnit Team. (n.d.). JUnit 5 user guide. <https://junit.org/junit5/docs/current/user-guide/>
- Martin, R. C. (2003). Agile software development: Principles, patterns, and practices. Prentice Hall.
- Mockito. (n.d.). Mockito framework. <https://site.mockito.org/>
- Spring Framework documentation. (n.d.). Spring Data JPA reference documentation. VMware. <https://spring.io/projects/spring-data-jpa>
- Testcontainers. (n.d.). Testcontainers for Java. <https://java.testcontainers.org/>
- Walls, C. (2022). Spring in action (6th ed.). Manning Publications.

ภาคผนวก

ภาคผนวก ก รายการ RESTful API Endpoint ทั้งหมด

ตารางที่ 16 รายการ RESTful API Endpoint ทั้งหมด

HTTP Method	Endpoint	คำอธิบาย
POST	/api/v1/auth/register	สมัครสมาชิกใหม่ด้วยอีเมล/รหัสผ่าน
POST	/api/v1/auth/login	เข้าสู่ระบบด้วยอีเมล/รหัสผ่าน
POST	/api/v1/auth/google	เข้าสู่ระบบด้วย Google ผ่าน Firebase
GET	/api/v1/users/me	ดูข้อมูลโปรไฟล์ของตนเอง
PUT	/api/v1/users/me	แก้ไขข้อมูลโปรไฟล์
POST	/api/v1/reports	สร้างประกาศแจ้งของหาย/พบของ
GET	/api/v1/reports	ค้นหาประกาศ พร้อม Pagination/Sorting
GET	/api/v1/reports/{id}	ดูรายละเอียดประกาศ
PATCH	/api/v1/reports/{id}/close	ปิดประกาศ
POST	/api/v1/reports/{id}/watch	ติดตามประกาศ
DELETE	/api/v1/reports/{id}/watch	เลิกติดตามประกาศ
POST	/api/v1/reports/{id}/claims	ยื่นคำขอเคลม
GET	/api/v1/claims/me	ดูคำขอเคลมของตนเอง
PATCH	/api/v1/claims/{id}/approve	อนุมัติคำขอเคลม (เจ้าของประกาศ)
PATCH	/api/v1/claims/{id}/reject	ปฏิเสธคำขอเคลม (เจ้าของประกาศ)
DELETE	/api/v1/admin/reports/{id}	ลบประกาศที่ไม่เหมาะสม (Staff)
GET	/api/v1/categories	ดูรายการหมวดหมู่
POST	/api/v1/categories	เพิ่มหมวดหมู่
GET	/api/v1/tags	ดูรายการแท็ก
POST	/api/v1/tags	เพิ่มแท็ก

HTTP Method	Endpoint	คำอธิบาย
GET	/api/v1/notifications	ดูรายการแจ้งเตือน
PATCH	/api/v1/notifications/{id}/read	ทำเครื่องหมายอ่านแล้ว
POST	/api/v1/uploads	อัปโหลดไฟล์รูปภาพ
GET	/api/v1/files/{id}	เรียกดูไฟล์รูปภาพ

ภาคผนวก ข ผลการทดสอบ API ทั้ง 22 กรณี

ตารางที่ 17 ผลการทดสอบ API ทั้ง 22 กรณี

รหัส	Method	Endpoint	วัตถุประสงค์	Status ที่ ได้	ผล
API-01	POST	/api/v1/auth/register	สมัครสมาชิกใหม่	201 Created	ผ่าน
API-02	POST	/api/v1/auth/login	เข้าสู่ระบบและรับ Token	200 OK	ผ่าน
API-03	POST	/api/v1/auth/google	เข้าสู่ระบบผ่าน Google/Firebase	200	ผ่าน
API-04	GET	/api/v1/users/me	ดูโปรไฟล์ผู้ใช้ปัจจุบัน	200	ผ่าน
API-05	PUT	/api/v1/users/me	แก้ไขโปรไฟล์ผู้ใช้	200	ผ่าน
API-06	POST	/api/v1/reports	สร้างประกาศของหาย/ของ พบ	201 Created	ผ่าน
API-07	GET	/api/v1/reports	ดูรายการประกาศ (Pagination/Sorting)	200 OK	ผ่าน
API-08	GET	/api/v1/reports/{id}	ดูรายละเอียดประกาศ	200 OK	ผ่าน
API-09	POST	/api/v1/reports/{id}/watch	ติดตามประกาศ	200 OK	ผ่าน
API-10	DELETE	/api/v1/reports/{id}/watch	เลิกติดตามประกาศ	204	ผ่าน
API-11	POST	/api/v1/reports/{id}/claims	ยื่นคำขอเคลม	201	ผ่าน
API-12	GET	/api/v1/claims/me	ดูคำขอเคลมของตนเอง	200	ผ่าน
API-13	PATCH	/api/v1/claims/{id}/approve	อนุมัติคำขอเคลม (เจ้าของ ประกาศ)	200	ผ่าน
API-14	DELETE	/api/v1/admin/reports/{id}	ลบประกาศโดย Staff	204	ผ่าน
API-15	GET	/api/v1/categories	ดูรายการหมวดหมู่	200	ผ่าน
API-16	POST	/api/v1/categories	เพิ่มหมวดหมู่	201	ผ่าน
API-17	GET	/api/v1/tags	ดูรายการแท็ก	200	ผ่าน
API-18	POST	/api/v1/tags	เพิ่มแท็ก	201	ผ่าน
API-19	GET	/api/v1/notifications	ดูการแจ้งเตือนของผู้ใช้	200	ผ่าน
API-20	PATCH	/api/v1/notifications/{id}/read	ทำเครื่องหมายว่าอ่านแล้ว	200	ผ่าน
API-21	POST	/api/v1/uploads	อัปโหลดไฟล์/รูปภาพ	201	ผ่าน
API-22	GET	/api/v1/files/{id}	เปิด/ดาวน์โหลดไฟล์	200	ผ่าน